МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)

Институт информационных технологий
Кафедра управления и информатики в технических системах

ЛАБОРАТОРНАЯ РАБОТА №1
ПО ДИСЦИПЛИНЕ
«АЛГОРИТМЫ И СТРУКТУРЫ ДАННЫХ»

СТУДЕНТА 2 КУРСА БАКАЛАВРИАТА ГРУППЫ ИДБ-25-11
(уровень профессионального образования)
Князева Глеба Романовича

ТЕМА РАБОТЫ

«Сортировка слиянием списков. Сравнение времени выполнения со стандартной функцией qsort и пузырьковой сортировкой»

Направление: 09.03.03 Прикладная информатика
Профиль подготовки: —

Отчет сдан «_____» _____ 2026 г.
Оценка _____
Преподаватель Евдокимов С.А., доцент
(Ф.И.О., должность, степень, звание)
(подпись)

МОСКВА 2026

Содержание

1. Сортировка слиянием списков 3
1.1 Постановка задачи 3
1.2 Описание алгоритма 4
2. Реализация программы и пример работы 5
3. Графики времени сортировки 10
4. Выводы 11
5. Литература 12

1. Сортировка слиянием списков

1.1 Постановка задачи

Написать программу сортировки слиянием списков по алгоритму из книги Д. Кнута «Искусство программирования», том 3, раздел 5.2.4 (сортировка слиянием со списками). Сравнить время её выполнения со стандартной функцией qsort и с простой пузырьковой сортировкой (алгоритм B из раздела 5.2.2 той же книги).

Для сравнения вычисляется время выполнения функций сортировки на массивах целых чисел следующих размеров: 16, 100, 500, 1000, 5000.

Исходный несортированный массив считывается из файла с именем d<число элементов>.txt. Имя входного файла задаётся в исходном коде до компиляции программы (строка с объявлением константы fileName).

Время работы функции сортировки рассчитывается с точностью до микросекунд с помощью std::chrono::steady_clock. Между вызовами функций подсчёта времени никаких операций вывода не выполняется, так как printf имеет большое собственное время работы и исказило бы замер.

Для оценки поведения функций сортировки создаются графики в зависимости от числа элементов (N) в массиве:

— график функции N^2;

— график функции $N \cdot \log_2 N$;

— график времени работы стандартной функции qsort;

— график времени работы функции сортировки слиянием списков;

— график времени работы пузырьковой сортировки.

Графики создаются в табличном процессоре по данным таблицы замеров.

1.2 Описание алгоритма

1. qsort — стандартная библиотечная функция сортировки. Ей передаются адрес массива, число элементов, размер одного элемента и функция сравнения. Типичная ожидаемая сложность эффективных библиотечных реализаций составляет $O(N \cdot \log_2 N)$, однако конкретные детали зависят от реализации библиотеки.

2. sortListMerge — сортировка слиянием со списками (раздел 5.2.4 книги Кнута). Признак данного метода состоит в том, что записи не перемещаются: вместо перестановки элементов используется вспомогательный массив связей links[0..n+1]. Отрицательное значение связи означает «конец списка» (приём "| |" по Кнуту). В начале работы каждый элемент массива рассматривается как одноэлементный список. Затем выполняются проходы слияния: на каждом проходе пары списков сливаются и распределяются обратно в исходный массив связей, пока в нём не останется один список. Слияние двух списков ведётся указателями p и q: меньший из текущих элементов включается в итоговый список, при этом функция setAbsL устанавливает связь по модулю со знаком, соответствующим концу списка. После завершения всех слияний элементы переписываются во временный массив и обратно в исходный — уже в порядке следования по связям. Трудоёмкость метода составляет $O(N \cdot \log_2 N)$ во всех случаях; дополнительная память — $O(N)$ под массивы связей и временный массив.

3. sortBubble — пузырьковая сортировка (алгоритм B из раздела 5.2.2). Алгоритм многократно сравнивает соседние элементы и меняет их местами, если они расположены в неправильном порядке; после каждого прохода наибольший элемент неотсортированной части перемещается к её концу. В программе используется модификация: запоминается позиция последнего обмена lastSwap, и следующий проход выполняется только до этой границы (bound), что сокращает число сравнений на частично упорядоченных данных. Средняя и худшая временная сложность — $O(N^2)$.

2. Реализация программы и пример работы

Программа состоит из основного управляющего блока (функция main) и трёх функций сортировки с единой сигнатурой void (int*, int). Программа считывает массив из входного файла, после чего функция runSort для каждого метода: копирует исходный массив original в рабочий массив arr, запускает сортировку и фиксирует её длительность. После сортировки выполняется проверка результата: если какой-либо элемент массива больше следующего, программа выводит сообщение об ошибке, в противном случае — измеренное время в микросекундах.

Запись алгоритма сортировки слиянием списков посредством диаграмм Насси-Шнейдермана приведена на рисунке 1.

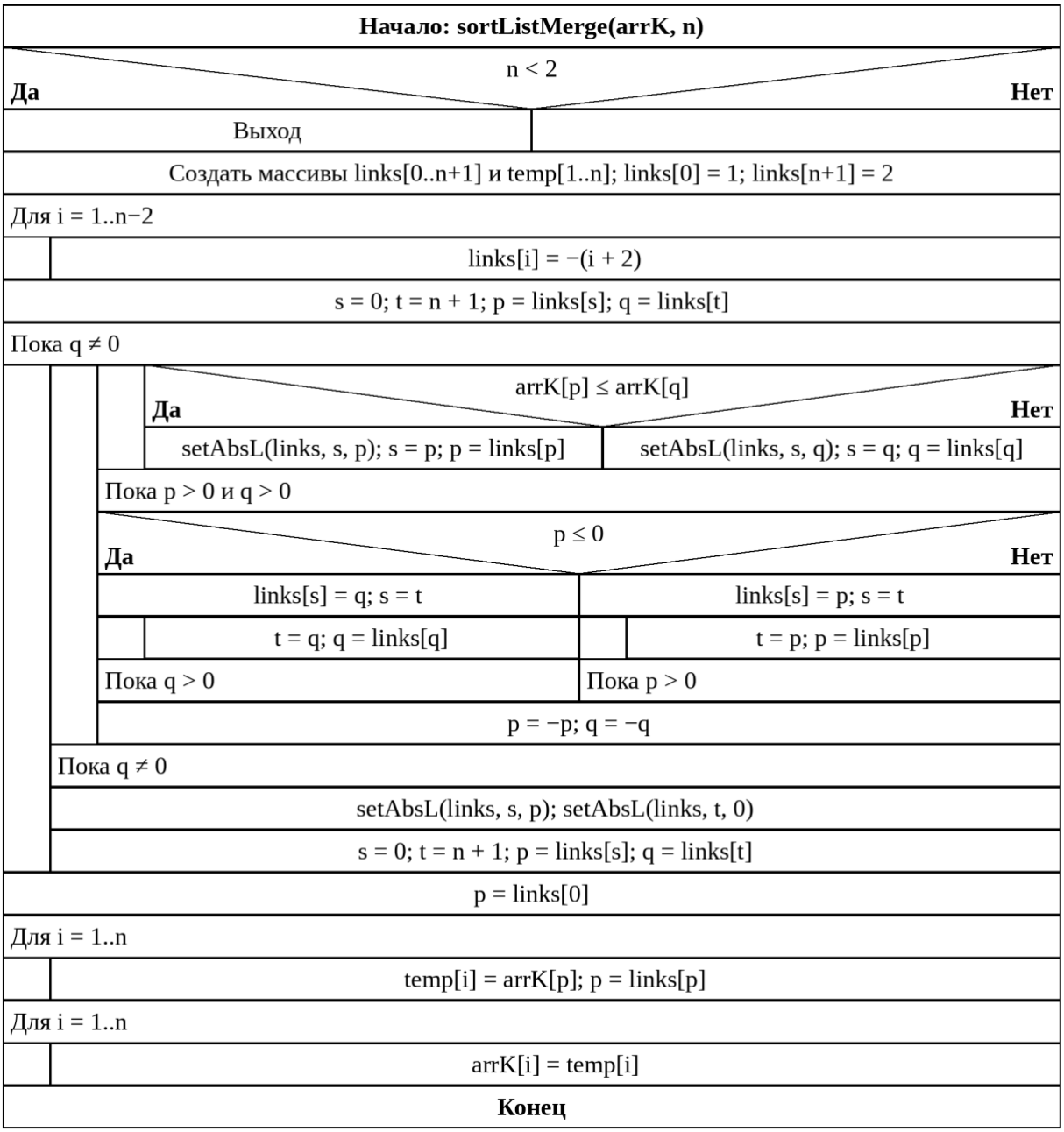


Рисунок 1 — Запись алгоритма сортировки слиянием списков посредством диаграмм Насси-Шнейдермана

Пример работы программы для файла d5000.txt:

```
Read 5000 numbers
qsort: 359 microseconds
list merge: 316 microseconds
bubble: 36240 microseconds
```

Исходный текст программы на C/C++:

```
#define _CRT_SECURE_NO_WARNINGS

#include <cstdio>
#include <cstdlib>
#include <chrono>
#include <vector>
```

```
const int MAX_N = 5000;
// const char* fileName = "d16.txt";
// const char* fileName = "d100.txt";
// const char* fileName = "d500.txt";
// const char* fileName = "d1000.txt";
const char* fileName = "d5000.txt";


int original[MAX_N + 1];
int arr[MAX_N + 1];
int N = 0;

int compareInts(const void* a, const void* b)
{
	int x = *(int*)a;
	int y = *(int*)b;
	return (x > y) - (x < y);
}



// Начало qSort
void sortWithQsort(int* arrK, int n)
{
	qsort(arrK + 1, n, sizeof(int), compareInts);
}
// конец qSort



void setAbsL(std::vector<int>& links, int s, int p)
{
	links[s] = links[s] < 0 ? -p : p; // "| |" по Кнуту
}



// Начало сортировки слиянием списков
void sortListMerge(int* arrK, int n)
{
	if (n < 2)
		return;

	std::vector<int> links(n + 2, 0);
	std::vector<int> temp(n + 1, 0);

	int s, t, p, q;

	links[0] = 1;
	links[n + 1] = 2;

	for (int i = 1; i <= n - 2; i++)
		links[i] = -(i + 2);

	while (true)
	{
		s = 0;
		t = n + 1;
		p = links[s];
		q = links[t];

		if (q == 0)
```

```
                break;

            do
            {
                do
                {
                    if (arrK[p] <= arrK[q])
                    {
                        setAbsL(links, s, p);
                        s = p;
                        p = links[p];
                    }
                    else
                    {
                        setAbsL(links, s, q);
                        s = q;
                        q = links[q];
                    }
                } while (p > 0 && q > 0);

                if (p <= 0)
                {
                    links[s] = q;
                    s = t;

                    do
                    {
                        t = q;
                        q = links[q];
                    } while (q > 0);
                }
                else
                {
                    links[s] = p;
                    s = t;

                    do
                    {
                        t = p;
                        p = links[p];
                    } while (p > 0);
                }

                p = -p;
                q = -q;

            } while (q != 0);

            setAbsL(links, s, p);
            setAbsL(links, t, 0);
        }

        p = links[0];

        for (int i = 1; i <= n; i++)
        {
            temp[i] = arrK[p];
            p = links[p];
        }

        for (int i = 1; i <= n; i++)
            arrK[i] = temp[i];
    }
```

```
// Конец сортировки слиянием списков



// Начало пузырька
void sortBubble(int* arrK, int n)
{
	int bound = n;

	while (bound > 1)
	{
		int lastSwap = 0;

		for (int i = 1; i < bound; i++)
		{
			if (arrK[i] > arrK[i + 1])
			{
				int temp = arrK[i];
				arrK[i] = arrK[i + 1];
				arrK[i + 1] = temp;
				lastSwap = i;
			}
		}

		bound = lastSwap;
	}
}
// Конец пузырька



// Общая функкция для всех сортировок
void runSort(
	const char* name,
	void (*sortFunction)(int*, int)
)
{
	for (int i = 1; i <= N; i++)
		arr[i] = original[i];

	auto start = std::chrono::steady_clock::now();
	sortFunction(arr, N);
	auto end = std::chrono::steady_clock::now();

	long long time =
		std::chrono::duration_cast<std::chrono::microseconds>(
			end - start
		).count();

	for (int i = 1; i < N; i++)
	{
		if (arr[i] > arr[i + 1])
		{
			printf("%s: error\n", name);
			return;
		}
	}

	printf("%s: %lld microseconds\n", name, time);
}
```

```
int main()
{
	FILE* file = fopen(fileName, "r");

	if (!file)
	{
		printf("Cannot open file\n");
		return 1;
	}

	while (N < MAX_N && fscanf(file, "%d", &original[N + 1]) == 1)
		N++;

	fclose(file);

	if (N == 0)
	{
		printf("No data\n");
		return 1;
	}

	printf("Read %d numbers\n", N);

	runSort("qsort", sortWithQsort);
	runSort("list merge", sortListMerge);
	runSort("bubble", sortBubble);

	return 0;
}
```

3. Графики времени сортировки

Для оценки поведения функций сортировки созданы графики в зависимости от числа элементов (N) в массиве. Данные для построения графиков приведены в таблице 1.

Таблица 1 — Данные для создания графиков времени сортировки

N	**N^2**	**$N \cdot \log_2 N$**	**qsort, мкс**	**Сортировка слиянием списков, мкс**	**Пузырьковая сортировка, мкс**
16	256	64	1	4	<1
100	10000	664	6	7	17
500	250000	4483	31	28	399
1000	1000000	9966	63	54	1532
5000	25000000	61439	359	316	36240

Замечание: для N = 16 время пузырьковой сортировки меньше разрешения таймера (в выводе программы — 0 мкс), в таблице оно приведено как <1, а на графике эта точка не показана.

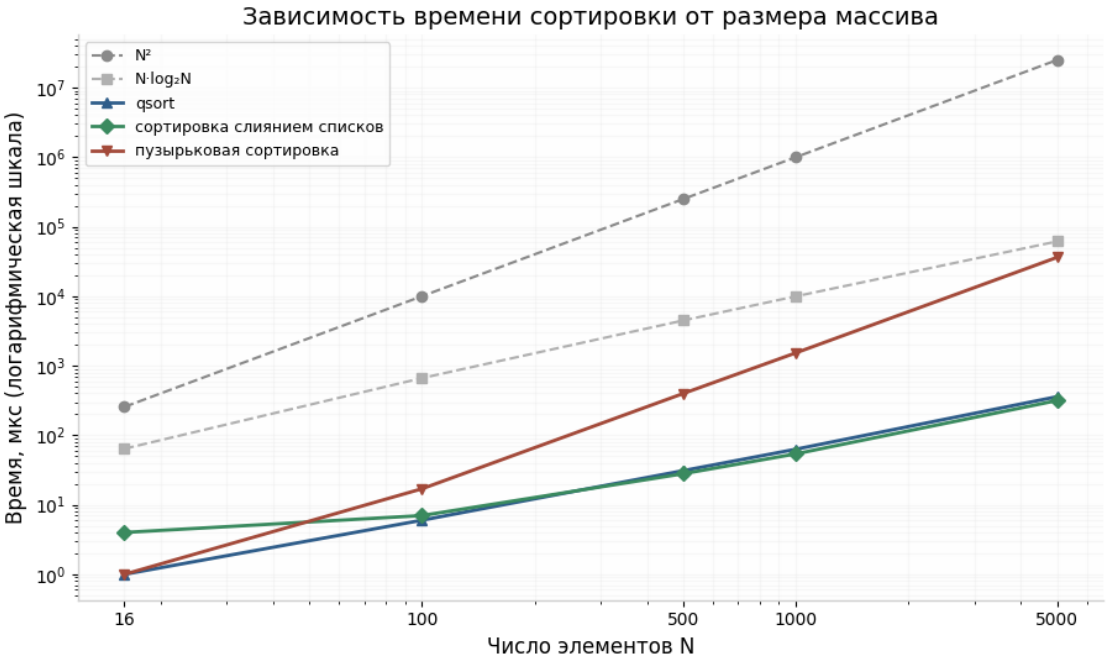


Рисунок 2 — Графики времени сортировки различными алгоритмами (логарифмические шкалы по осям X и Y)

4. Выводы

Из анализа графиков на рисунке 2 следует:

1) Разработана программа на C++, позволяющая считывать массив целых чисел из файла, выполнять три метода сортировки, измерять время их работы и проверять правильность результата.

2) Исследуемый метод sortListMerge реализует сортировку слиянием со списками по Кнуту: элементы массива не перемещаются, вместо этого ведётся массив связей со значениями-признаками концов списков, слияние выполняется распределением обратно в исходный массив, после чего элементы переписываются в порядке следования по связям. Трудоёмкость метода — $O(N \cdot \log_2 N)$ во всех случаях.

3) По полученным измерениям сортировка слиянием списков и стандартная функция qsort работают сопоставимо и значительно быстрее пузырьковой сортировки на массивах размера 500–5000. Пузырьковая сортировка при росте размера массива резко замедляется, что согласуется с её квадратичной сложностью $O(N^2)$.

4) Для более надёжного сравнения рекомендуется выполнять несколько серий испытаний, использовать одинаковые входные данные, проверять результат после каждой сортировки и отдельно учитывать время выделения памяти.

5. Литература

1. Кнут Д. Искусство программирования для ЭВМ. Т. 3: Сортировка и поиск. — М.: Мир, 1978.